



Final Term Public.

Midterm (10 題選 3 題)

Final (5 題中選 3 題)

1. Write a simple C++ program to demonstrate **Run-Time Polymorphism**. You should explain your program. (10%)
↳ virtual function
2. Explain what **abstract classes** are, and write an abstract class in C++. (10%)
3. Write a simple C++ program to demonstrate **Class Template**. You should explain your program. (10%)
4. Explain what **higher-order functions** are, and write a higher-order function in C++. (10%)
5. Write a simple C++ program to demonstrate the use of **Lambda Expression**. You should explain your program. (10%)

1.

```
class Animal{
public:
    virtual void sounds(){
        cout<<"some sounds"<<endl;
    }
};
```

```
class Dog : public Animal{
public:
    void sounds() override{
        cout<<" woof ! " <<endl;
    }
};
```

```
class Cat : public Animal{
public:
    void sounds() override{
        cout<<" meow ! " <<endl;
    }
};
```

```
int main(){
    Animal *a;
    Dog b;
    Cat c;
    a = &b;
    a->sounds() // Output  Woof!
    a = &c;
    a->sounds() // Output  meow!
    return 0;
}
```

先在父類別 Animal 中宣告一 virtual function sounds, 讓繼承 Animal 的子類別能夠 override。

接著在子類別 Cat 和 Dog 分別實作 Sound 函式, 最後宣告一 Animal 指標、指向任一子類別並呼叫 sounds 函式, 程式在執行時便會去呼叫指定的 Sounds 函式, 此即 Run-Time Polymorphism.

2.

抽象類別 (Abstract Classes) 指類別中 至少有一個 pure virtual function 的 class。這種類別 無法被實例化。抽象類別通常被當作 其他類別的基底 (父類別), 而繼承其的子類別必須要實作父類別定義的 pure virtual function。

```
class Shape{
public:
    virtual int calArea()=0;
}
};
```

```
class Rect : public Shape{
    int h, w;
    Rect(int hIn, int wIn){h=hIn, w=wIn;}
    int calArea() override{
        return h*w;
    }
};
```

```
int main(){
    Rect a(5,7);
    cout<< a.calArea() <<endl;
    return 0;
}
```

→ Shape 是一 abstract classes, 定義了一 pure virtual function 供子類別實作。

→ 子類別 Rect 繼承自 Shape, 並實作 calArea()。

3.

```
template <typename T>
class plus{
public:
    T add(T a, T b){
        return a+b;
    }
};

int main(){
    plus<int> plusint;
    plus<double> plusdouble;
    cout << plusint.add(6, 6) << endl //12
        << plusdouble.add(4.1, 5.1) << endl; //9.2
    return 0;
}
```

class Template plus<T> 能夠允許為任何資料
形態建立此物件，而函式 add 則會將傳入的兩變數
相加。

4.

Higher-order function 指的是以一個函數作為參數的函式。

```
int square(int x){ return x*x; }

void used(int x, int (*func)(int)){ // Higher-Order Function
    cout << func(x) << endl;
}

int main(){
    used(6, square); // Output 36
    return 0;
}
```

函式 used 是一個 Higher-Order Function，
他可以接受一個函式指標作為參數，並將其
回傳結果轉型成整數後輸出。

Function Object (Functor)

ex. `int operator()(int x) const{`
 `return x;`
 `}`
 * overloading () 運算子

5.

```
int main(){
    vector<int> v{9, 8, 6, 1, 7, 4, 7, 5, 2};
    sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
    for (auto i: v) cout << i << endl;
    return 0;
}
```

使用匿名函式來要求 std::sort 由大到小來排序

Introduction to Programming (II) Midterm Exam
ID:

Name: 黃翊宏
01351101

Student

1. How many lines will the program print? (2%) What is the output of the program? (6%) Explain why the variable j cannot be initialized in line 8. (2%)

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     for (int i = 9; i <= 13; i++) {
5.         if (i % 2 == 0 || i < 11)
6.             continue;
7.         int j = 8;
8.         for (; j >= 5; j--) {
9.             if (j < 7)
10.                break;
11.            cout << (j^i) << endl;
12.        }
13.        cout << (double)(i+j) << endl;
14.    }
15. }
```

Handwritten calculations and notes for Question 1:

- Line 8: j cannot be initialized because it is modified in the same scope (line 11).
- Output for $i=9$: 9^9 (1 line)
- Output for $i=10$: 10^{10} (1 line)
- Output for $i=11$: 11^{11} (1 line)
- Output for $i=12$: 12^{12} (1 line)
- Output for $i=13$: 13^{13} (1 line)
- Sum of $i+j$ for $i=9$ to 13 : $9+8, 10+7, 11+6, 12+5, 13+4$ (5 lines)

2. Please describe the advantage and disadvantage of **inline function**. (5%) What kind of functions are suitable to be written as **inline functions**? (5%)
3. Write a simple C++ program to demonstrate **"function overloading"**. You should also explain your program. (10%)
4. Write a simple C++ program to demonstrate **"function template"**. You should also explain your program. (10%)
5. What is the output of the following program? (5%) What problem does the following program have? You should explain the problem in detail. (5%)

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     int p = 6;
5.     int *q = new int;
6.     int *r = new int;
7.     *q = p;
8.     *r = 8;
9.     r = q;
10.    *r += (*q + p);
11.    cout << *q << endl;
12.    cout << *r << endl;
13. }
```

Handwritten calculations and notes for Question 5:

- Initial state: $p=6, q=6, r=8$
- After $r=q$: $p=6, q=6, r=6$
- After $*r += (*q + p)$: $p=6, q=6, r=12$
- Output: 6 (from $*q$), 12 (from $*r$)

Handwritten calculations for Question 5:

```

144
x 12
-----
288
144
-----
1728
    
```

6. What is the output of the program below?

```
1. #include <iostream>
2. using namespace std;
3. int f(int a) {
4.     if (a <= 5)
5.         return 12;
6.     cout << a << endl;
7.     return f(a-2)*f(a-1);
8. }
9. int main() {
10.    cout << f(8) << endl;
11. }
```

Handwritten calculations and notes for Question 6:

- Recursion tree for $f(8)$: $f(8) \rightarrow f(6) \rightarrow f(4) \rightarrow f(2) \rightarrow f(0)$
- Base case: $f(0) = 12$
- Calculation: $f(2) = 12, f(4) = 12 * 12 = 144, f(6) = 144 * 12 = 1728, f(8) = 1728 * 144 = 248832$

Handwritten calculations for Question 6:

```

144
x 12
-----
288
144
-----
1728
    
```

7. Explain what the difference between the **smart pointer** and the **raw pointer** is. (10%)
8. Write three programs to demonstrate **Call by Value**, **Call by Address**, and **Call by**

Handwritten code for Question 8:

```

return (a+b)/15;
}

double avg(double a, double b) {
    return (a+b)/2;
}
    
```

return 0;

Handwritten notes for Question 8:

- 不同, 因此編
- 確的區分他們
- overloading.

Reference in C++. You should explain the programs you write. (10%)

9. What is the output of the program below? (10%)

```
1. #include <iostream>
2. using namespace std;
3. int f(int c[3]) {
4.     c[2] = 8;
5.     return c;
6. }
7. int main() {
8.     int a[3] = {5, 2, 6};
9.     int *b;
10.    b = f(a);
11.    b[1] = a[0]+a[1];
12.    a[0] = *(b+2);
13.    for (int x: a)
14.        cout << x << endl;
15. }
```

Handwritten analysis for Q9:
 a = 8 7 8
 b = 5 2 6
 (Arrows indicate array modification: a[2] becomes 8, b[1] becomes a[0]+a[1]=7, a[0] becomes b[2]=6)

10. (1) Figure out the lines that will cause compilation error. (5%) (2) After removing the lines that will cause compilation error, what is the output of this program? (5%)

```
1. #include <iostream>
2. using namespace std;
3. class Midterm {
4. private:
5.     int i1;
6.     static int si1;
7. public:
8.     int i2;
9.     Midterm(int v) {
10.         i1 = v+1;
11.         i2 = v+3;
12.         si1 = v+5;
13.     }
14.     static int si2;
15.     static void setVariables(int x) {
16.         i1 = v+1;
17.         si2 = x+2;
18.     }
19. };
20.
21. int Midterm::si1 = 2;
22. int Midterm::si2 = 3;
23.
24. int main(int argc, const char * argv[]) {
25.
26.     Midterm midterm1(2);
27.     Midterm midterm2(3);
28.     midterm1.setVariables(5);
29.
30.     cout << Midterm::si1 << endl;
31.     cout << Midterm::si2 << endl;
32.
33.     midterm1.si2 = 5;
34.
35.     cout << midterm1.i1 << endl;
36.     cout << midterm2.i1 << endl;
37.     cout << midterm1.i2 << endl;
38.     cout << midterm2.i2 << endl;
39.     cout << midterm1.si1 << endl;
40.     cout << midterm2.si1 << endl;
41.     cout << midterm1.si2 << endl;
42.     cout << midterm2.si2 << endl;
43. }
```

Handwritten notes for Q10:
 midterm1
 private: i1: 3

public: i2: 5

midterm2
 private: i1: 4
 public: i2: 6

Static private: si1: 2

public: si2: 3

Handwritten notes:
 ✓ Static Member Function
 不能使用到unstatic Member Variables.

Cannot Access Private Variables!

```
int avg (int a, int b, int c) {
    return (a+b+c)/3;
}

double avg (double a, double b) {
    return (a+b)/2;
}
```

avg(4.26, 8.96);
 return 0;

雖然都叫作
 時參數的
 不同，因此
 確的區分化
 overloading.

1.

3
1 2
1
5
1 0
2

6 Lines will the program print.
若在第8行初始化j, j就会变成
Line 8 ~ Line 12 for 迴圈的區塊
變數, 但因 Line 13 还需要利用到j,
所以這時就必須把j在第7行先初
始化, Line 13 才可以使用到j。

3.

```
int avg (int a, int b){ return (a+b)/2; } //1
int avg (int a, int b, int c){ return (a+b+c)/3; } //2
double avg (double a, double b){ return (a+b)/2; } //3
int main(){
    avg(2, 6);
    avg(4, 2, 6);
    avg(4.26, 9.87);
    return 0;
}
```

function 1 是对 2 int 取平均,
function 2 是对 3 int 取平均,
function 3 对 2 double 取平均。

虽然都叫 avg, 但其在呼叫時參數个数、形態、
回傳形態皆不同, 因此編譯器能夠準確的
区分他們, 此即 function over-loading.

5.

L7 *q → P(6)

L8 *r → 8

L9 *q → P(6)

↑
*r → 8 ← 失去此位置的控制權!
Memory Leaked!

L10 *r += *q + P(6) ⇒ *r = 18, *q = 18

Output:

18
18

2

Advantage

会在函式的呼叫点將被呼叫的函式展
開插入, 省去了呼叫堆疊時的傳輸、延遲
時間, 讓呼叫函式變快。

Disadvantage

因 inline function 是將被呼叫的函式展開
插入, 所以会导致編譯後的二進制檔變
大、編譯時間較長。

→ 較精簡的函式較適合被寫成
inline function, 例如:

```
inline int add (int a, int b){
    return (a+b);
}
```

4.

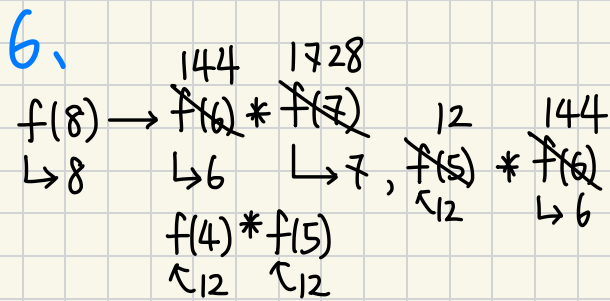
```
template <typename T>
T add (T a, T b){ return a+b; }
// Can Accept int, double, float, ...
```

```
int main(){
    add(9, 10);
    add(4.26, 9.87);
}
```

add 函式在做的是兩數相加, 但因其是
泛型, 所以 add 可以接受各種不同類型的
參數, 如 int、float、double 等。

function Template!

在第9行時, 指標r改指向與指標q相同的
位置, 導致原本指標r所指的位置失去
控制權, 進而造成記憶體洩露。



7. Smart Pointer: 内部会维护一个 Reference Count, 当不再被引用、或变数离开作用域后会自动释放, 较安全且 memory Leaked 的风险降低。

Raw Pointer: 传统指标所有的维护都依靠 Programmer 处理, 因此很可能会发生安全性问题, 如忘记释放或重复释放, 进而造成記憶體洩露。不过, 因为没有经过物件封装, 效能会比 Smart Pointer 好。

8.

```

int pValue (int a){
    a += 1;
}

int pAddr (int *a){
    *a += 1;
}

int pRefer (int &a){
    a += 1;
}
  
```

```

int main(){
    int x=5;
    pValue(x);
    pAddr(&x);
    pRefer(x);
    return 0;
}
  
```

pValue 是 Call by Value, 会将变数 x 複製一份到 pValue 函数中再 +1, 不影响到 main 中 x 的值。

pAddr 是 Call by Address, 将变数 x 的指标传给 pAddr 函数再 +1, 由于是传指标给函数所以 pAddr 加一的操作会影响到 main 中 x 的值。

pRefer 是 Call by Reference, 将变数 x 的参考传给 pRefer 函数后再 +1, 所以 pRefer 加一的操作也会影响到 main 中 x 的值。

9.

$a = 8, 7, 8$
 $b = \uparrow$
 Output: 8, 7, 8

10.

Line 16: Static Member Function 不可以使用到 unstatic member Variables.

Line 30, Line 35-36, Line 39-40 在 Class 外不可以使用 Private member variables.

Midterm1
private
i1: 3
public
i2: 5

Midterm2
private
i1: 4
public
i2: 6

Static
private
s1: 7, 8
public:
s2: 5, 5

Output:
7
5
6
5
5

Final Term 考古

Introduction to Programming (M) Final Term Exam Name: 林冠宇 Student ID: 01257027

1. Write a simple C++ program to demonstrate how "short circuit evaluation" works. You should also explain your program. (10%)
2. Write a simple C++ program to demonstrate "function overloading". You should also explain your program. (10%)
3. Write a simple C++ program to explain the "memory leak" problem. You should also explain your program. (10%)
4. Explain what **abstract classes** are, and write an abstract class in C++. (10%)
5. Explain what **higher-order functions** are, and write a higher-order function in C++. (10%)
6. Write a simple C++ program to demonstrate the use of **Lambda Expression**. You should also explain your program. (10%)
7. What is the output of the following program? (10%)

```
#include <iostream>
using namespace std;
class Shape {
public:
    double area;
    virtual double calculateArea() { return 0.0; };
    void printArea() { cout << "(Shape) " << area << endl; };
};
class Square: public Shape {
public:
    double side;
    double calculateArea() { return side * side; };
    void printArea() { cout << "(Square) " << area << endl; };
};
void f(Shape & s) {
    s.area = s.calculateArea();
    s.printArea();
}
int main(int argc, const char * argv[]) {
    Square square;
    square.side = 3.0;
    f(square);
    square.printArea();
    Shape shape = square;
    f(shape);
    shape.printArea();
    Shape *sp = &square;
    f(*sp);
    sp->printArea();
}
```

area = 9.0 0.0

✓
not ✓!

virtual double calculateArea() { return 0.0; };
void printArea() { cout << "(Shape) " << area << endl; };
class Square: public Shape {
public:
 double side; 3.0, area = 9.0
 double calculateArea() { return side * side; };
 void printArea() { cout << "(Square) " << area << endl; };
};
void f(Shape & s) {
 s.area = s.calculateArea();
 s.printArea();
};
int main(int argc, const char * argv[]) {
 Square square;
 square.side = 3.0;
 f(square);
 square.printArea();
 Shape shape = square;
 f(shape);
 shape.printArea();
 Shape *sp = □
 f(*sp);
 sp->printArea();
};

Run-Time Polymorphism! (Square::calculateArea())
not virtual!, 利用參考形別!

Call
CP Time
已知

把 Area copy!
(Side won't be copy!)

等價 f(square)
Area 已根上層調為 9.0

8. What is the output of the following program? (10%)

```
#include <iostream>
using namespace std;
class TwoNumbers {
public:
    int x; int y;
    TwoNumbers(int x, int y) {this->x = x+1; this->y = y;}
    TwoNumbers operator^(const TwoNumbers & tn) const {
        return TwoNumbers(x-tn.x, y*tn.y);
    }
};
int main(int argc, const char * argv[]) {
    TwoNumbers tn1(6, 2); TwoNumbers tn2(5, 3);
    TwoNumbers tn3 = tn1 ^ tn2;
    cout << tn3.x << ", " << tn3.y << endl;
}
```

tn1 7, 2

tn2 6 3

(7,2)^(6,3)

7-6=1
2*3=6

1, 6

9. What is the output of the following program? (10%)

```
#include <iostream>
using namespace std;
template<class T, class S>
class Check {
public:
```

```
    bool isEqual(T data1, S data2) {
        if (data1 == (T)data2) {
            return true;
        } else {
            return false;
        }
    }
};
```

```
int main() {
    cout << "boolalpha";
    int i1 = 6, i2 = 8, i3 = 6;
    double d1 = 6.3, d2 = 8.6;
    Check<int, double> c1;
    cout << c1.isEqual(i1, d1) << endl; true
    Check<double, int> c2;
    cout << c2.isEqual(d1, i1) << endl; false
    Check<int *, int *> c3;
    cout << c3.isEqual(&i1, &i3) << endl; false
}
```

→ casting 到 T

以 True, False 呈现!

10. (1) Figure out the lines that will cause compilation error, (2) After removing the lines that will cause compilation error, what is the output of this program? (10%)

```
01 #include <iostream>
02 using namespace std;
03 class A {
04 private:
05     int a1;
06 protected:
07     int a2;
08 public:
09     int a3;
10     A(int value) { a1 = value+6; a2 = value+7; a3 = value+8; }
11     A(const A &a) { a1 = a.a1+2; a2 = 1; }
12     int fun1() { return a1; }
13 };
```

```
14 class B: public A {
15 private:
16     int b1;
17 protected:
18     int b2;
19 public:
20     B(): A(6) { b1 = 3; b2 = 5; }
21     int fun1() { return A::fun1(); }
22     int fun2() { return (b1, a1); }
23     int fun3() { return (b2+a2); }
24 };
```

```
25 int main(int argc, const char * argv[]) {
26     A a = A(7);
27     B b1;
28     B b2 = b1;
29     cout << a.a3 << endl;
30     cout << b1.a3 << endl;
31     cout << b2.a3 << endl;
32     cout << a.fun1() << endl;
33     cout << b1.fun1() << endl;
34     cout << b2.fun1() << endl;
35     cout << a.fun3() << endl;
36     cout << b1.fun3() << endl;
37     cout << b2.fun3() << endl;
38 }
```

default

copy constructor

$a.a1 = 13$
 $a.a2 = 14$
 $a.a3 = 15$
 $b1.b1 = 3$
 $b1.b2 = 5$
 $b2.a1 = 12$
 $b2.a2 = 13$
 $b2.a3 = 14$

15

29

30

31

32

33

34

35

36

37

38

(x)

$13+5=18$
 $5+1=6$

1. (不考)

```
#include <iostream>
using namespace std;
```

```
bool rF(){
    cout << "rF is called\n";
    return false;
}
```

```
bool rT(){
    cout << "rT is called\n";
    return true;
}
```

AND 的特性: 有 false 就一定 false

OR 的特性: 有 true 就一定 true

所以, 在 AND Test 時只有 rF 被呼叫, 在 OR Test 時只有 rT 被呼叫。

因為結果已確定, 剩下 2 個呼叫被短路。

```
int main(){
```

```
    // AND Test
```

```
    if (rF() && rT()){
```

```
        cout << "AND Result is T\n";
```

```
    } else cout << "AND Result is F\n";
```

```
    // OR Test
```

```
    if (rT() || rF()){
```

```
        cout << "OR Result is T\n";
```

```
    } else cout << "OR Result is F\n";
```

Output:

rF is called

AND Result is F

rT is called

OR Result is T

2.

```
int avg (int a, int b){ return (a+b)/2; } // 1
```

```
int avg (int a, int b, int c){ (a+b+c)/3; } // 2
```

```
double avg (double a, double b){ return (a+b)/2; } // 3
```

```
int main(){
```

```
    avg(4, 6);
```

```
    avg(4, 2, 6);
```

```
    avg(8.7, 7.8);
```

```
    return 0;
}
```

3 個 function 都是取平均, 但因傳入的參數形態、個數皆不同, 因此呼叫時編譯器能夠準確的区分他們, 此即 function overloading

function
overloading!

3.

```
int main(){
```

```
    int *p = new int(6);    p → 6
```

```
    int *q = new int(4);    q → 4
```

```
    p = q;
```

```
    return 0;
```

```
}
```

```
p → 6
q → 4
```

原本指標 q 所指向的地址失去控制權，進而造成記憶體洩露。

4.

抽象類別 (Abstract Classes) 指 Class 中至少有一個 pure virtual function。這種類別無法被實例化。抽象類別通常作為其他類別的基底 (父類別)，而繼承其的子類別必須要實作父類別定義的 pure virtual function。

```
class Shape{
```

```
    public:
```

```
        virtual int calArea()=0;
```

```
};
```

```
class Rect{
```

```
    public:
```

```
        int h, w;
```

```
        Rect(int hin, int win){h=hin, w=win;}
```

```
        int calArea() override {
```

```
            return h * w;
```

```
        }
```

```
};
```

5.

High-Order Function 指的是以一個函式做為參數 Function.

```
int square(int x){return x*x;}
int used(int x, int (*func)(int)){ // High-Order Function
    cout << func(x) << endl;
}
int main(){
    used(6, square); // Output 36
    return 0;
}
```

6.

```
int main(){
    vector<int> v{7,8,9,5,3,6,1,2,4};
    sort(v.begin(), v.end(), [](int a, int b){return a>b;});
    for(auto i: v) cout << i << endl;
    return 0;
}
```

使用匿名函式來要求 std::sort 由大到小來排序。

7.

(Shape) 9.0
(Square) 9.0
 (Shape) 0.0
 (Shape) 0.0
 (Shape) 9.0
 (Shape) 9.0

8.

x y
 tn 1: 7 , 2
 tn 2: 6 , 3
 → TwoNumbers(1, 6)
 tn 3 = 2 , 6
 2 , 6

9.

True
 False
 False

10.

Line 22: class B 不能使用到 class A 的 private member variables.

Line 35: 類別 A 沒 fun3() 這個 function。

a:	b ₁ :	b ₂ :
a ₁ 13	b ₁ 3	b ₁ 3
a ₂ 14	b ₂ 5	b ₂ 5
a ₃ 15	a ₂ 13	a ₂ 1
	a ₃ 14	a ₃ ?
	(a ₁ 12)	(a ₁ 14)

15
 14
 ?
 13
 12
 14
 18
 6

